

KamaClaude Coding Agent 演进与量化优化路线

文档状态：持续维护

最近更新：2026-09-17

目的：统一记录 KamaClaude 向 Coding Agent 演进过程中的现状、设计决策、实施路线、评测方法和简历素材。

1. 项目定位

KamaClaude 当前是一个本地双进程 Agent Runtime：`kama-core` 负责会话、模型调用、工具执行、权限和事件持久化，CLI/TUI 通过 JSON-RPC NDJSON 与 Core 通信。

项目目标不是堆叠模型 API 或工具数量，而是构建一个具备以下特性的 Coding Agent：

- 能准确理解和搜索代码；
- 能以局部、可回滚的方式修改代码；
- 能自动运行测试并根据失败继续修复；
- 能在长会话和异常退出后恢复；
- 能安全执行命令并控制文件、网络边界；
- 能协调多个隔离的子 Agent；
- 所有行为都有事件、日志和指标，可以复现和量化。

2. 当前能力快照

2.1 已有基础能力

- ReAct 风格 Agent Loop；
- Anthropic 流式模型调用；
- 文件读取、目录遍历、文件写入、Bash 工具；
- Task 创建、更新、查询；
- 权限审批和持久化策略；
- EventBus、`events.jsonl`、trace 和 TUI 实时展示；
- Skills、Agent Profile 和 MCP；
- 前台、后台及嵌套子 Agent；
- Session、多轮对话、notes 和上下文压缩。

2.2 Session 消息图与压缩

当前 Session 已从线性、覆盖式消息文件改造成 append-only 消息图：



核心语义：

- 每个消息节点都有 `id`、`parent_id`；
- Session 使用 `current_id` 表示当前 Head；
- 普通消息和 compact 都写入统一的 `thread.jsonl`；
- compact 只追加一个摘要节点，不覆盖、不删除旧消息；
- assistant、单个 tool result 和 compact 在逻辑节点形成后立即追加、flush 并 `fsync`，不再等待整个 Run 结束；
- 同一步的多个 tool result 在物理日志中保持独立节点，恢复模型视图时再合并为一条 user 消息；
- 恢复时从 `current_id` 沿 `parent_id` 回溯；
- 遇到最近的 compact 节点后停止；
- 旧格式 JSONL 会获得稳定的虚拟 ID，保持向后兼容；
- 该结构为后续消息分支和 checkout 提供基础。
- Daemon 启动时扫描并恢复磁盘 Session，旧格式会话会迁移 `current_id`；
- 异常退出遗留的 `active` 状态会改为 `interrupted`，恢复后可以继续对话；
- 若进程退出在 JSONL 刷盘后、meta Head 更新前，启动恢复会将 `current_id` 安全推进到同一 Run 的唯一子节点；
- 若中断发生在 tool use 已持久化、tool result 尚未产生之间，恢复会追加合成错误结果以重新配平模型消息；
- 损坏会话逻辑隔离，不会阻止其他 Session 和 Daemon 启动。
- Session 元数据记录客户端启动目录的规范化绝对路径 `workspace_root`；
- `session.list` 只列出当前工作区内可恢复的 chat，`session.resume` 按 ID 加载有效消息链；
- Session 简介默认取首条用户消息的前 30 个字符，并归一化换行和连续空白；
- CLI `kama resume` 展示“序号 + 简介 + 状态 + 时间”并用序号选择，不要求用户接触 Session ID；
- TUI 的 `/resume` 使用可聚焦列表，支持方向键或 `j/k` 移动、Enter 确认和 Esc 取消；`/resume <number|ID>` 保留兼容；
- 退出 TUI/CLI 不再关闭 chat，旧版已关闭 chat 也可在显式恢复后继续。
- `session.branch //branch [node_id]` 可从当前或历史节点创建新 Session；来源消息不复制，由 fork 元数据递归投影；
- 分支代码运行在 detached Git worktree 中，按 checkpoint 精确恢复 HEAD、index、worktree 和未跟踪文件，源工作区与分支后续修改互不影响；
- 成功的潜在写工具在 tool result 节点落盘后创建 `mutation` checkpoint；明确只读的 Bash/Git 查询不再触发快照，路径明确的编辑只纳入对应新文件；
- checkpoint 在 Git/LFS 读取内容前对变更文件数、单文件和总字节执行硬预算，超限时快速跳过并提示检查 `.gitignore`；取消或超时会终止 Bash/Git 的完整进程树；
- Core 通过统一 `RunManager` 按 `run_id` 管理顶层和子 Agent，支持工作区范围内查询、手动取消、父子级联取消及慢任务/无进展提醒；TUI 使用 `Ctrl+C` 取消当前 Run、`Ctrl+R` 选择任意活动 Run；
- 历史分支会沿完整父链穿过 compact 寻找最近 checkpoint，并检查 checkpoint 后是否存在未覆盖的成功写操作；旧会话状态不完整时明确拒绝“精确代码分支”，不会静默使用过期快照；

- checkpoint tree 通过 `refs/kama/checkpoints/*` 保留，避免长期 Session 的 tree object 被 Git GC 回收。

压缩行为：

- JSONL 永久保存完整 tool result；
- 只有发送给摘要模型的消息副本会按预算截断；
- 手动和自动压缩共用相同的 tool result 截断配置；
- prompt 明确要求把已有摘要和新增消息合并成独立、累积的新摘要；
- 当前 tool result 截断仍只保留前缀，后续应升级为头尾保留。

2.3 结构化代码搜索 `search_code`

当前已经加入专用 `search_code` 工具，支持：

- 固定字符串搜索，默认不会误解正则字符；
- 显式正则模式；
- 大小写控制；
- 文件名 glob 过滤；
- 全局最大结果数；
- `path:line:column:source` 稳定输出；
- 跳过二进制文件、超大文件和常见生成目录；
- 结果行长度限制，避免搜索输出占满上下文；
- 默认使用 Ripgrep JSON 流式后端，并按路径排序保证确定性；
- 达到全局结果上限后立即终止搜索子进程；
- Ripgrep 自动遵守 `.gitignore`，同时排除常见生成目录；
- Ripgrep 不存在、启动失败或不支持给定正则时自动回退 Python；
- Python fallback 在线程中执行，不阻塞 Agent 事件循环；
- Python fallback 先用 `stat` 判断大小，超限文件不读取内容；
- 支持通过 `.env` 追加文件名、后缀和 glob 忽略规则；
- 将 Ripgrep 的 UTF-8 字节偏移转换为字符列号；
- 权限系统将其识别为默认允许的只读工具；
- 根 Agent、Planner、Executor、Reviewer 和子 Agent 均可按角色使用。

它相对直接执行 `rg` 的主要价值不是当前搜索速度，而是：

1. 工具参数结构稳定，模型不需要处理 shell 转义；
2. 输出格式和最大规模可控；
3. 权限系统能区分只读搜索与任意 Bash；
4. 调用过程可独立审计、统计和调度；
5. Windows/Linux 行为更容易保持一致；
6. 为后续 AST、LSP 和语义检索后端提供统一接口。

当前局限：

- Python fallback 不完整解析 `.gitignore`，其 ignore 语义弱于默认 Ripgrep 后端；
- Python fallback 对未超限候选最多读取 1 MB + 1 字节，仍不是分块流式匹配；
- 每行只报告第一个匹配；
- 不支持前后文、跨行匹配、符号和引用关系；
- 结构化文件和搜索路径已统一经过 WorkspaceBoundary，但 Bash 仍需 OS sandbox 才能阻止命令自行访问绝对外部路径。

计划升级为：

```
SearchCodeTool
├─ TextSearch
│   └─ RipgrepBackend      # 默认高性能实现
│   └─ PythonBackend       # 跨平台 fallback
├─ SymbolSearch           # Tree-sitter/LSP 定义与引用
└─ SemanticSearch         # 后期可选，不作为前置依赖
```

2.3.1 文件读取边界与用户忽略规则

`read_file` 和 `search_code` 共用用户追加的文件忽略规则。配置在 daemon 启动时加载，修改后需要重启 daemon 才能生效。

用户要求的 `.env` 写法可直接使用：

```
ignore_file=['.env','.xx']
```

同时支持带项目前缀的等价写法：

```
KAMA_IGNORE_FILE=['.env','.xx']
```

也可以使用逗号分隔值：

```
KAMA_IGNORE_FILE=.env,.xx,*.pem,private/**
```

匹配语义：

规则	行为
<code>.env</code> 、 <code>.xx</code>	匹配文件名后缀，例如 <code>.env</code> 、 <code>settings.env</code> 、 <code>secret.xx</code>
<code>secrets.json</code>	匹配完整文件名
<code>*.pem</code>	使用 glob 匹配文件名
<code>private/**</code>	使用相对工作区路径 glob

安全与内存行为：

- 用户规则追加在内置目录过滤、二进制过滤、文件大小限制和 Ripgrep `.gitignore` 规则之上，不替换默认规则；
- `read_file` 对命中用户规则的文件抛出权限错误，不读取内容；
- `read_file` 先读取文件元数据，超过 512 KB 时返回错误，不再读取并截断整个大文件；

- Python 搜索后端先读取文件元数据，超过 1 MB 时直接跳过；
- 为处理 `stat` 后文件并发增长，实际读取函数最多读取“阈值 + 1 字节”，不会无界载入内存；
- Ripgrep 后端把相同用户规则转换为排除 glob，确保两个搜索后端行为一致。

也支持 TOML：

```
[files]
ignore_file = [".env", ".xx", "*.pem"]
```

2.4 精确局部编辑 `edit_file`

当前已经加入 `edit_file` 工具：

- 使用 `old_text` → `new_text` 精确替换；
- 默认要求旧文本只出现一次；
- 支持显式 `expected_replacements`；
- 出现次数不一致时返回 `conflict`，文件保持不变；
- 使用同目录临时文件、`flush`、`fsync` 和 `os.replace` 原子替换；
- 保留未参与修改的原始换行字节和文件 mode；
- 拒绝符号链接和基础父目录遍历；
- 权限策略默认为 `ASK`；
- 根 Agent 和执行型子 Agent 可以使用，Planner/Reviewer 保持只读。

它与 `search_code/read_file` 形成最小 Coding Agent 编辑链：

```
search_code → read_file → edit_file → bash/test
```

2.5 Git checkpoint/rollback

当前已经加入面向脏工作树的可恢复 checkpoint：

- 使用临时 `GIT_INDEX_FILE` 从真实 index tree 增量构造工作树 tree，兼容尚无首个 commit 但已有暂存文件的仓库；
- 已跟踪变化使用 `git add -u`，未跟踪文件只有通过数量/容量预算后才分批加入；内置编辑工具使用精确路径，不扫描无关数据集；
- 创建 checkpoint 不修改用户真实 index 和工作树；
- 分别保存真实 index tree 与完整 worktree tree，因此可精确重建 staged/unstaged 的差异；
- checkpoint 元数据保存在 `.git/kama-checkpoints/`，可关联 Session ID、消息节点 ID 和 Run ID；
- rollback 默认只返回文件级预览和 `expected_state` 确认令牌；
- apply 前重新计算 HEAD/index/worktree 状态，令牌过期或 HEAD 改变时拒绝恢复；
- apply 前自动创建 undo checkpoint，恢复后再次捕获 tree 并校验结果；
- `git_checkpoint` 默认允许，`git_rollback preview` 默认允许，apply 必须经过权限审批；
- 当前只向根 Agent 注册，避免共享工作树中的并行子 Agent 相互回滚。

当前边界：

- 只允许在相同 HEAD 内恢复，不支持跨提交 rollback；
- 遵守 Git ignore，ignored untracked 文件不会被 checkpoint 捕获或删除；
- tree object 和元数据尚无保留期限及垃圾回收策略；
- 尚未提供 checkpoint 列表、名称查询和 TUI 可视化；当前硬预算为内建默认值，后续可开放项目配置；
- 尚未实现子 Agent 独立 worktree，因而不向子 Agent 暴露恢复工具。

2.6 Project-aware VerificationManager

当前已经加入 `verify_project` 工具和独立 VerificationManager：

- 根据 `pyproject.toml`、`package.json`、`Cargo.toml`、`go.mod` 识别 Python、Node、Rust 和 Go；
- Python 根据 uv/Poetry 锁文件、测试目录、工具配置和依赖生成命令；
- Node 只使用 `package.json#scripts` 中实际存在的 test/lint/typecheck/build，并识别 npm/pnpm/yarn/bun；
- Rust 和 Go 使用标准工具链命令；
- 多生态计划统一按 test → lint → typecheck → build 排序；
- 默认 `run=false` 只返回可审查计划并自动放行，`run=true` 必须经过 ASK 权限审批；
- 命令使用参数数组直接启动，不经过 shell 解释；
- 每项检查独立超时，输出采用有界头尾保留，避免日志无界进入内存和模型上下文；
- 统一返回 kind、ecosystem、tool、command、status、exit code、elapsed、output、截断标记和 diagnostics；
- 通过 `ParserRegistry` 将 pytest、Ruff、Mypy、进程超时/启动错误解析为统一 `Diagnostic`，未知工具可降级提取 `path:line:column`；
- 诊断包含 category、severity、文件位置、规则码、test ID 和证据片段，便于后续自动修复、TUI 展示和指标统计；
- `verify_project` 默认省略已结构化或成功检查的重复原始日志，未知失败仍保留最多 4 KiB 头尾；调试时可用 `include_raw_output=true` 获取 Runner 的有界原始输出；
- 已建立 100 条受控生成日志的 KamaTestLogBench v1，真实执行 pytest/Ruff/Mypy，并保存源码、原始日志、Oracle 和 Parser 输出；
- 新增 `required_on_write` 完成门禁：成功执行 `edit_file`、`write_file` 或实际应用 `git_rollback` 后将 Run 标记为待验证；
- 模型尝试 `end_turn` 时若仍待验证，Runtime 会通过正常权限链自动注入一次 `verify_project(run=true)`，而不是只依赖提示词要求模型自觉测试；
- 验证失败的结构化诊断会保留在消息上下文中，供模型继续分析、修复并再次验证；验证通过或项目没有适用检查时才允许成功结束；
- 修复循环受最大验证次数、总耗时和 AgentLoop 最大步数共同限制，权限拒绝和预算耗尽会返回明确失败原因；
- 支持 `off`、`suggest`、`required_on_write` 三种模式，次数和总耗时可由 TOML 或环境变量配置；
- Runtime 在首次内置写操作执行前自动建立 `verification` checkpoint，避免把用户任务开始前的 dirty 状态混入本次增量范围；

- Python AST 测试索引持久化到 `.git/kama-test-index/index.json` 并绑定精确 worktree tree, 索引 tree 由 `refs/kama/test-index/current` 保留;
- 每次验证前比较旧索引 tree 与当前 tree, 只重解析新增、修改、删除或重命名的 Python 文件; 即使 Agent 离线期间用户修改了测试 import, 下一次验证也会自动刷新反向依赖图;
- 将任务 checkpoint 与当前 tree 的 diff 映射到 changed test、直接 import 和传递 import 相关测试, 并把 checkpoint ID、变更文件、选择原因和降级原因写入验证计划;
- 配置变化、非 Python 变更、全局 Python 文件、解析失败、无相关测试或索引期间工作区继续变化时安全降级全量测试;
- 验证失败属于业务结果, 不触发工具层的盲目自动重试;
- 支持 fail-fast, 并把未执行项显式标记为 skipped;
- 根 Agent、Executor 和 Reviewer 可以使用, Planner 保持纯规划角色。

当前边界:

- 增量测试已具备 Git/Python 静态 import 图和可插拔语义 diff 链路: `DiffAnalyzer` → `ParserRegistry` → `PythonParser` → `UnifiedSymbol` → `ChangeClassifier`; 当前仅 Python parser 实际启用, JS/TS、Go 和 coverage 生产选择仍待实现;
- TypeScript、Jest/Vitest、Cargo 和 Go 尚无专用解析器, 当前只能使用通用位置降级;
- 当前 Parser 消费 Runner 已截断至 32 KiB 的文本, 还没有优先采用 JSON/JUnit/SARIF 等机器格式, 也没有单独持久化完整测试日志;
- 尚未建立带人工标注的真实 CI 留出集; 当前 100% 准确率只代表固定版本、固定环境的受控生成语料;
- 当前变更跟踪只覆盖内置编辑/写入工具和实际应用的 Git rollback, 无法可靠识别 Bash、MCP 或外部进程产生的文件修改;
- 验证状态目前只存在于单次 Run 内, 尚未跨 Run/重启持久化, 也没有独立 token 预算;
- 模型的 `end_turn` 文本会先流式送达客户端, 再由完成门禁决定是否继续修复, 后续需要增加“验证后再确认最终答复”的 UI 状态;
- 项目自定义命令仍依赖标准清单和脚本名, 暂不解析自然语言项目指令;
- 超时会终止直接子进程, 但跨平台进程组和孙进程清理仍需统一 Sandbox 支持;
- 报告目前通过 tool result/Event 进入 run 记录, 尚无独立历史索引和 TUI 面板。

2.7 Workspace Session Resume 与路径边界

当前会话工作区由客户端启动目录确定, 而不是 daemon 的常驻目录:

```
client Path.cwd().resolve()
    ↓ workspace_root
session.create / agent.run
    ↓ meta.json
Runner / root Agent / sub Agent tools
    ↓
workspaceBoundary.resolve(path)
```

核心规则:

- `workspace_root` 必须是存在目录, 并统一规范化为绝对路径;
- 相对路径始终相对于该 Session 工作区, 而非 daemon 当前目录;

- 绝对路径只有在工作区内部才允许；
- 真实路径解析后再次检查边界，因此工作区内指向外部的符号链接无法逃逸；
- `read_file`、`write_file`、`edit_file`、`list_dir`、`search_code`、Git 和 Verification 工具共用同一边界；
- Bash 子进程的 `cwd` 固定为工作区，但完整读写隔离仍依赖后续 OS 级 sandbox；
- 根 Agent 与子 Agent 共享同一工作区边界；
- Git 仓库根也必须在工作区内，避免从嵌套目录访问上级仓库的其他文件；
- 历史旧 Session 没有可靠的 `cwd` 信息，不会自动混入任意工作区列表；知道 ID 时可首次绑定当前工作区。

恢复流程：

1. TUI/CLI 把启动目录的绝对路径发送给 daemon；
2. daemon 按 `workspace_root` 过滤可恢复 chat；
3. 前端展示序号、首条消息简介、状态和更新时间，用户选择序号；
4. `session.resume` 校验工作区归属并从 `current_id` 恢复活动链；
5. 旧版 `closed` chat 显式恢复为 `waiting_for_input`，one-shot 任务不可恢复；
6. 后续每个 Run 继续使用 Session 中已持久化的工作区。

2.8 Agent Evaluation Harness

当前已加入面向真实 Coding Agent 任务的可复现评测入口：

- `kama eval <suite.json> --repeats N` 按任务矩阵串行执行并生成汇总报告；
- 每次运行固定 `base_ref` 对应 commit，在系统临时目录创建 detached Git worktree，原工作区不接收 Agent 修改；
- Agent 使用现有 `AgentRunner`、工具、自动验证和事件链路，Harness 额外执行任务定义中的无 shell oracle；
- 任务支持模型、最大步骤、工具白名单、Agent 总超时和单项验证超时；
- EventBus 聚合步骤、工具调用/失败、权限请求、输入/输出/cache Token 和总耗时；
- 使用有界 Git tree 快照捕获 tracked/untracked 变化，并保存二进制 patch 和 changed files；
- 每次运行保存 request、result、metrics、verification、events 和 patch，suite 汇总成功率、验证通过率及耗时 p50/p95；
- 正常、失败和超时路径都会回收 worktree，显式 `--keep-worktrees` 才保留调试现场；
- 评测运行不创建用户 Session，不进入 `/resume` 列表；默认关闭不可复现的全局 `~/.kama/context.md` 注入，但保留仓库内 `.kama/context.md`；
- 当前隔离边界是 Git worktree，不是 OS sandbox；Bash 仍需后续执行后端限制文件系统、网络和资源。

2.9 当前代码实现导航

下表按能力列出主要实现入口和对应测试。阅读时建议先看“入口”，再顺着 import 和调用关系进入“核心实现”。

能力	主要实现文件	相关测试
Core 启动与依赖组装	src/kama_claude/core/app.py	tests/unit/test_core_app_signals.py 、集成测试位于 tests/integration/

能力	主要实现文件	相关测试
Agent Run 依赖和工具注册	src/kama_claude/core/runner.py	tests/unit/test_runner.py
Run 注册、监控与取消	src/kama_claude/core/run_manager.py 、 src/kama_claude/core/app.py 、 src/kama_claude/core/bus/commands.py 、 src/kama_claude/core/bus/events.py 、 src/kama_claude/tui/app.py	tests/unit/test_run_manager.py 、 tests/unit/test_tui_app.py 、 tests/unit/test_commands_events.py
Agent 主循环	src/kama_claude/core/loop.py	tests/unit/test_loop.py
单次执行上下文和待持久化消息	src/kama_claude/core/context.py	tests/unit/test_context.py 、 tests/unit/test_context_system_prompt.py
Session 数据模型	src/kama_claude/core/session/model.py	tests/unit/test_session_store.py
Session 生命周期、启动恢复与手动压缩入口	src/kama_claude/core/session/manager.py 、 src/kama_claude/core/app.py	tests/unit/test_session_manager.py
工作区边界与符号链接逃逸防护	src/kama_claude/core/workspace.py 、 src/kama_claude/core/runner.py	tests/unit/test_workspace.py 、 tests/unit/test_runner.py
Session 列表/恢复协议及前端入口	src/kama_claude/core/bus/commands.py 、 src/kama_claude/core/app.py 、 src/kama_claude/tui/app.py 、 src/kama_claude/cli/commands/chat.py	tests/unit/test_session_manager.py 、 tests/unit/test_tui_app.py 、 tests/integration/test_s4_session_ipc.py
Session 历史分支与独立代码状态	src/kama_claude/core/session/manager.py 、 src/kama_claude/core/session/model.py 、 src/kama_claude/core/session/store.py 、 src/kama_claude/core/git/worktree.py 、 src/kama_claude/core/git/checkpoint.py 、 src/kama_claude/core/verification/controller.py	tests/unit/test_session_branch.py 、 tests/unit/test_git_checkpoint.py
append-only JSONL、消息图恢复与完整性校验	src/kama_claude/core/session/store.py	tests/unit/test_session_store.py
累积摘要和自动压缩	src/kama_claude/core/compact/compactor.py	tests/unit/test_compactor.py
tool result 截断策略	src/kama_claude/core/compact/budget.py	tests/unit/test_budget.py
LLM 抽象、响应类型和 Anthropic 实现	src/kama_claude/core/llm/base.py 、 src/kama_claude/core/llm/types.py 、 src/kama_claude/core/llm/provider.py	tests/unit/test_llm_provider.py
工具抽象与注册表	src/kama_claude/core/tools/base.py 、 src/kama_claude/core/tools/registry.py	tests/unit/test_tool_registry.py 、 tests/unit/test_tool_params.py
工具校验、权限检查、重试和事件	src/kama_claude/core/tools/invoke.py	tests/unit/test_invoke.py 、 tests/unit/test_tool_retry.py
结构化代码搜索	src/kama_claude/core/tools/builtin/search_code.py	tests/unit/test_search_code.py
文件忽略规则与配置传播	src/kama_claude/core/tools/file_filter.py 、 src/kama_claude/core/config.py 、 src/kama_claude/core/runner.py 、 src/kama_claude/core/subagent/tool.py	tests/unit/test_config_env.py 、 tests/unit/test_read_file.py 、 tests/unit/test_search_code.py
精确局部编辑	src/kama_claude/core/tools/builtin/edit_file.py	tests/unit/test_edit_file.py
结构化 Git 状态	src/kama_claude/core/tools/builtin/git_status.py 、 src/kama_claude/core/tools/builtin/_git.py	tests/unit/test_git_status.py
有界 Git diff	src/kama_claude/core/tools/builtin/git_diff.py 、 src/kama_claude/core/tools/builtin/_git.py	tests/unit/test_git_diff.py
Git 子进程边界	src/kama_claude/core/git/process.py 、兼容入口 src/kama_claude/core/tools/builtin/_git.py	tests/unit/test_git_status.py 、 tests/unit/test_git_diff.py 、 tests/unit/test_git_checkpoint.py
Git checkpoint/rollback	src/kama_claude/core/git/checkpoint.py 、 src/kama_claude/core/tools/builtin/git_checkpoint.py 、 src/kama_claude/core/tools/builtin/git_rollback.py	tests/unit/test_git_checkpoint.py 、 tests/unit/test_permission_policy.py 、 tests/unit/test_runner.py
项目验证计划、执行与失败解析	src/kama_claude/core/verification/detector.py 、 src/kama_claude/core/verification/manager.py 、 src/kama_claude/core/verification/runner.py 、 src/kama_claude/core/verification/parser.py 、 src/kama_claude/core/verification/model.py 、 src/kama_claude/core/tools/builtin/verify_project.py	tests/unit/test_verification.py 、 tests/unit/test_verification_parser.py 、 tests/unit/test_permission_policy.py 、 tests/unit/test_runner.py 、 tests/unit/test_agent_profile_loader.py
自动验证完成门禁与有界修复循环	src/kama_claude/core/verification/controller.py 、 src/kama_claude/core/loop.py 、 src/kama_claude/core/runner.py 、 src/kama_claude/core/config.py	tests/unit/test_verification_controller.py 、 tests/unit/test_config_env.py 、 tests/integration/test_verification_auto_loop.py
持久化测试结构索引与增量 pytest 选择	src/kama_claude/core/verification/test_index.py 、 src/kama_claude/core/verification/semantic/diff_analyzer.py 、 src/kama_claude/core/verification/semantic/python_parser.py 、 src/kama_claude/core/verification/semantic/classifier.py 、 src/kama_claude/core/verification/manager.py 、 src/kama_claude/core/git/checkpoint.py 、 src/kama_claude/core/tools/builtin/verify_project.py 、 src/kama_claude/core/session/manager.py 、 src/kama_claude/tui/app.py	tests/unit/test_verification_test_index.py 、 tests/unit/test_semantic_diff.py 、 tests/integration/test_verification_auto_loop.py 、 tests/unit/test_tui_app.py
增量 pytest mutation 评测	benchmarks/benchmark_incremental_tests.py 、 benchmarks/incremental_tests/pytest_recorder.py 、 benchmarks/incremental_tests/coverage_exporter.py 、 benchmarks/results/incremental_tests/windows_20_2026-09-16.json 、 benchmarks/results/incremental_tests/coverage_4case_2026-09-16.json	tests/unit/test_incremental_benchmark.py 、 tests/unit/test_verification_test_index.py
Agent Evaluation Harness	src/kama_claude/core/harness/ 、 src/kama_claude/cli/commands/eval.py 、 examples/evaluation_suite.example.json	tests/unit/test_evaluation_harness.py

能力	主要实现文件	相关测试
测试日志数据生成与 Parser 评测	benchmarks/generate_test_log_dataset.py 、 benchmarks/datasets/test_logs_v1/summary.json 、 benchmarks/results/test_log_parser_windows_100_2026-09-15.json	tests/unit/test_test_log_dataset.py 、 tests/unit/test_verification_parser.py
其他内置文件和 Bash 工具	src/kama_claude/core/tools/builtin/	tests/unit/test_builtin_tools.py 、 tests/unit/test_read_file.py
权限决策和持久化	src/kama_claude/core/permissions/manager.py 、 src/kama_claude/core/permissions/policy.py 、 src/kama_claude/core/permissions/storage.py	tests/unit/test_permission_manager.py 、 tests/unit/test_permission_policy.py
子 Agent 创建、结果获取和任务注册	src/kama_claude/core/subagent/tool.py 、 src/kama_claude/core/subagent/registry.py	tests/unit/test_spawn_agent_tool.py
Planner/Executor/Reviewer 配置	src/kama_claude/core/agents/builtin/ 、 src/kama_claude/core/agents/loader.py	tests/unit/test_agent_profile_loader.py
Skills 发现和内置工作流	src/kama_claude/core/skills/loader.py 、 src/kama_claude/core/skills/builtin/	tests/unit/test_skill_loader.py
Task 状态管理	src/kama_claude/core/task/manager.py 、 src/kama_claude/core/task/model.py	tests/unit/test_task_manager.py 、 tests/unit/test_task_model.py
MCP 客户端、服务管理和工具适配	src/kama_claude/core/mcp/	tests/unit/test_mcp_tool.py
事件模型、事件文件和内部总线	src/kama_claude/core/bus/events.py 、 src/kama_claude/core/events/bus.py 、 src/kama_claude/core/events/writer.py	tests/unit/test_commands_events.py 、 tests/unit/test_event_bus.py 、 tests/unit/test_event_writer.py
Trace 记录和 LLM 调用观测	src/kama_claude/core/trace/	tests/unit/test_trace_writer.py 、 tests/unit/test_tracing_provider.py
JSON-RPC 与事件广播	src/kama_claude/core/transport/ 、 src/kama_claude/core/bus/commands.py	tests/unit/test_socket_server.py 、 tests/unit/test_socket_client.py 、 tests/unit/test_ipc_broadcaster.py
TUI 展示和交互	src/kama_claude/tui/app.py	tests/unit/test_tui_app.py
配置与项目上下文	src/kama_claude/core/config.py 、 src/kama_claude/core/memory/loader.py	tests/unit/test_config_env.py 、 tests/unit/test_memory_loader.py

3. 常见 Coding Agent 能力矩阵

能力	当前状态	下一步
文件读取与目录遍历	已有大文件预检和用户追加忽略规则	支持按行、offset 和上下文读取
代码文本搜索	Ripgrep 流式后端与 Python fallback 已完成	上下文行、多命中、统一 WorkspaceBoundary
精确局部编辑	基础版本已完成	多 edit 事务、统一 diff、失败回滚
Git diff/status/log	结构化 status/diff 已完成，log 可通过 Bash	后续增加结构化 log 和 checkpoint 关联
Checkpoint/rollback	写后状态点、Git ref 保留、只读 Bash 跳过、LFS 前预算和进程树取消已完成	增加列表/TUI、保留期限和清理策略
Worktree 隔离	Session 历史分支已使用独立 worktree	扩展到并发子 Agent、生命周期清理和容量配额
自动验证与修复	项目识别、失败解析、受控评测、有界修复循环、Git/Python 静态和语义测试选择已完成；4 项目 20 mutation 初测完成	收益门禁、真实 CI 留出集、coverage 关系、其他生态、跨 Run 状态与独立 token 预算
Agent Evaluation Harness	基础版已完成：固定 commit、隔离 worktree、外部 oracle、patch/事件/指标与 suite 汇总	并行调度、环境镜像、随机种子、模型矩阵、OS sandbox 和 Web/TUI 报告

能力	当前状态	下一步
项目指令发现	部分支持 <code>.kama/context.md</code>	用户/项目/子目录分层规则和惰性加载
长会话恢复	Daemon 启动恢复、状态迁移、消息级落盘和损坏隔离已完成	诊断/修复 API、偏移索引
消息实时持久化	assistant/tool/compact 逻辑节点已实时追加	流式 assistant 草稿与故障注入压测
Context 预算	支持 compact	调用模型前预检与模型输入投影
权限审批	已有	与 OS sandbox、域名白名单结合
OS 级 Sandbox	未实现	文件系统、网络和资源强制隔离
Hooks	只有内部 EventBus	可阻断、修改和扩展的生命周期 Hooks
Run/子 Agent 生命周期	统一 run_id 注册、查询、顶层/子 Agent 级联取消、慢任务提醒已完成	跨重启持久化、并发限制、历史状态索引
Parallel Tool Calls	模型可返回多个，当前顺序执行	只读并行、写操作串行、依赖调度
AST/LSP 代码理解	未实现	symbol、definition、references、diagnostics
Diff-aware Review	有 Reviewer Profile	增加 diff、严重级别、调用关系和验证证据
GitHub/CI 集成	未实现	Issue→任务→PR、CI 失败分析
浏览器/视觉验证	未实现	前端任务截图、DOM/Console 检查
模型路由与成本预算	配置中只有静态路由	任务分类、fallback、预算和质量策略

3.1 后续功能的建议代码落点

以下路径是当前设计建议，标注为“新增”的文件尚未创建，实施时可以根据实际边界调整。

后续功能	主要修改位置	建议新增位置	重点测试
daemon 启动恢复 Session（已完成）	<code>core/app.py</code> 、 <code>core/session/manager.py</code> 、 <code>core/session/store.py</code> 、 <code>core/session/model.py</code>	无	已覆盖状态恢复、继续对话、旧格式迁移、坏会话隔离和消息图损坏；待补真实 Daemon 重启集成测试
消息生成时实时追加 （已完成）	<code>core/context.py</code> 、 <code>core/runner.py</code> 、 <code>core/session/store.py</code> 、 <code>core/session/manager.py</code>	后续可新增 <code>core/session/sink.py</code>	已覆盖 Loop 内落盘、逐工具结果合并、未配对 tool use 修复和 durable orphan Head 快进

后续功能	主要修改位置	建议新增位置	重点测试
Session 崩溃一致性评测 (基础版已完成)	<code>core/app.py</code> 、 <code>core/session/store.py</code> 、 <code>core/session/manager.py</code>	<code>benchmarks/benchmark_session_crash.py</code>	真实 daemon 覆盖 7 个写入阶段，集成测试执行硬退出与二次重启；待扩展 compact 和随机长序列
消息图校验与修复	<code>core/session/store.py</code>	新增 <code>core/session/integrity.py</code>	missing parent、cycle、duplicate ID、broken tail
Session 分支与 checkout (基础版已完成)	<code>core/session/model.py</code> 、 <code>core/session/store.py</code> 、 <code>core/session/manager.py</code> 、 <code>core/bus/commands.py</code> 、 <code>core/app.py</code> 、TUI	已新增 <code>core/git/worktree.py</code>	已覆盖当前/历史节点、compact 边界、旧 checkpoint 写入间隙、消息继承和代码隔离；待补会话树浏览、清理与配额
模型输入预算化投影	<code>core/loop.py</code> 、 <code>core/compact/budget.py</code> 、 <code>core/runner.py</code>	新增 <code>core/context_builder.py</code>	大 tool result、调用前 compact、不修改 canonical history
Ripgrep 搜索后端 (已完成)	<code>core/tools/builtin/search_code.py</code>	后续可拆为 <code>core/search/text.py</code> 、 <code>core/search/rg.py</code>	已覆盖 fallback、输出上限、 <code>.gitignore</code> 、Unicode 列号；待补显式取消测试
AST/LSP 代码理解	<code>core/runner.py</code> 、工具注册和配置	新增 <code>core/code_intelligence/</code> 及 <code>symbol/reference</code> 工具	定义、引用、rename、语言服务异常
多文件事务编辑	<code>core/tools/builtin/edit_file.py</code> 、权限策略	新增 <code>core/edit/transaction.py</code> 、 <code>core/edit/diff.py</code>	部分失败回滚、并发冲突、换行/权限保留
Git status/diff (已完成)	<code>core/runner.py</code> 、权限策略、子 Agent 和角色配置	已新增 <code>core/tools/builtin/git_status.py</code> 、 <code>git_diff.py</code> 、 <code>_git.py</code>	已覆盖 dirty/untracked/rename/staged/binary/输出上限/非仓库
Checkpoint/rollback (基础版已完成)	<code>core/runner.py</code> 、权限策略；后续接 Session/TUI	已新增 <code>core/git/checkpoint.py</code> 、 <code>git_checkpoint.py</code> 、 <code>git_rollback.py</code>	已覆盖用户混合修改、精确恢复、undo、过期令牌、HEAD 变化；待补 GC 与故障注入
VerificationManager、有界修复与增量 pytest (基础版已完成)	<code>core/runner.py</code> 、 <code>core/loop.py</code> 、 <code>core/config.py</code> 、 <code>core/git/checkpoint.py</code> 、权限策略和 Agent Profile	已新增 <code>core/verification/test_index.py</code> 、 <code>semantic/</code> parser 层、 <code>controller.py</code> 、 <code>parser.py</code> 和 <code>verify_project.py</code>	已覆盖任务前 checkpoint、离线索引刷新、直接/传递 import、Python 函数体语义缩小、全量降级、失败修复重试和完成门禁；待补 coverage、其他生态、选择器 benchmark 和独立 token 预算
OS Sandbox	<code>core/tools/invoction.py</code> 、权限配置	新增 <code>core/sandbox/</code>	文件越界、网络访问、子进程继承、资源上限
生命周期 Hooks	<code>core/events/bus.py</code> 、 <code>core/tools/invoction.py</code> 、Session/Compactor	新增 <code>core/hooks/</code>	deny/modify/timeout/多个 Hook 优先级
子 Agent 跨 run 持久化	<code>core/subagent/registry.py</code> 、 <code>core/subagent/tool.py</code> 、 <code>core/runner.py</code> 、 <code>core/app.py</code> 、 <code>core/run_manager.py</code>	新增 <code>core/subagent/store.py</code>	当前已支持在线查询/取消和级联清理；待补重启恢复、并发配额和历史索引
Parallel Tool Scheduler	<code>core/loop.py</code> 、 <code>core/tools/base.py</code> 、 <code>core/tools/invoction.py</code>	新增 <code>core/tools/scheduler.py</code>	只读并行、写入串行、依赖失败、取消传播
Worktree 隔离 (Session 分支已完成)	Runner、子 Agent、权限和 Session 元数据	已新增 <code>core/git/worktree.py</code>	当前已覆盖 Session 分支恢复；待补并发 Agent、生命周期清理、容量配额和故障注入
Diff-aware Review	Agent Profile、Skills、Runner、TUI	新增 <code>core/review/</code>	严重级别、误报率、行号、相关测试验证
GitHub/CI 集成	MCP/工具注册、配置、权限	新增对应 MCP adapter 或 <code>core/integrations/github/</code>	权限、幂等、PR/评论失败恢复
Benchmark 与 KamaBench	无生产代码依赖	新增 <code>benchmarks/</code>	固定数据、随机种子、baseline/optimized 对比

4. 已发现的优先问题

已解决：daemon 重启后无法真正恢复已有 Session

`SessionManager` 现在会在 Daemon 开始监听前扫描 Session 目录，恢复合法元数据和锁。磁盘中的 `active` 状态会持久化转换为 `interrupted`，用户可以继续发送消息；`closed/waiting` 状态保持不变。

已经增加：

- `SessionStore.list_session_ids()`；
- `SessionManager.load_existing_sessions()` 和结构化恢复报告；
- 启动时恢复 Session 和 lock；
- 将异常退出遗留的 `active` 状态转换为 `interrupted`；
- 严格检查重复 ID、missing parent、parent cycle、错误 `current_id` 和破损 JSON；
- 旧格式无 ID 节点的稳定虚拟 ID 与 `current_id` 迁移；

- 单个损坏 Session 逻辑隔离，不阻塞 Daemon 启动。

Session 列表、工作区恢复和消息级实时追加现已完成；仍需增加真实 Daemon 重启故障注入、诊断修复命令和大型 JSONL 偏移索引。

已解决：run 中间消息没有实时进入 thread

`ExecutionContext` 现在通过同步 append sink，在 assistant、每个 tool result 和 compact 形成后立即写入 `thread.jsonl`；Runner 末尾仅对未接入 Session sink 的剩余节点做兼容补写，避免重复追加。

存储顺序是“JSONL 追加并 `fsync` → 原子更新 meta `current_id`”。启动恢复会识别同一 Run 中唯一的 durable orphan，并为未配对 tool use 追加合成错误结果。完整 assistant 响应形成之前的流式文本仍不作为 canonical 节点保存。

P1：完整存储视图和模型输入视图没有彻底分离

存储层返回完整 tool result 是正确的，但主模型也可能直接收到全部历史。自动 compact 又依赖上一次模型调用的 `context_pctx`，巨大工具结果可能在下一次调用前就造成 context overflow。

建议增加 `ModelContextBuilder`：

- 存储恢复始终完整；
- 每次模型调用前生成预算化副本；
- 超预算时先 compact；
- 新旧 tool result 采用相同策略；
- 优先保留 tool 输出头部和尾部；
- 不修改 canonical messages。

P1：后台子 Agent 注册表不是真正跨 run 共享

`BackgroundTaskRegistry` 当前属于单个 `AgentRunner`，而每次 Session 消息都会创建新的 Runner。后台任务跨越父 run 后，下一轮可能无法通过 `agent_result` 找到。

建议：

- registry 提升到 SessionManager/CoreApp；
- 按 session 管理任务；
- 记录 queued/running/success/failed/cancelled；
- 结果和错误持久化；
- daemon 退出时统一取消；
- 完成任务定期淘汰；
- 重启后明确标记 interrupted。

P1：history API 混合了真实历史和模型投影

compact 被投影为一条 user summary 和一条 synthetic assistant ack。该形式适合模型调用，但不适合 TUI 展示和分支操作。

建议拆分：

- `read_active_nodes()`：返回真实节点；

- `build_model_messages()`: 构建模型消息;
- `session.get_history`: 返回包含 ID、parent ID 和 kind 的真实 DTO;
- TUI 单独渲染 compact 节点。

P2: 消息图完整性诊断与显式修复 API 仍不足

需要检查:

- ID 是否唯一;
- `current_id` 是否存在;
- parent 是否存在;
- 是否存在 parent cycle;
- 节点 kind、role、content schema 是否合法;
- JSONL 尾部是否是不完整写入;
- meta 更新前崩溃产生的同 Run 唯一孤儿节点已自动恢复; 分叉孤儿仍需显式诊断和人工选择。

损坏时应返回结构化错误, 而不是把空历史交给模型继续运行。修复操作必须显式执行并保留审计记录。

5. 实施路线

M0: 整理当前变更边界

- 将 Session/compact 改造整理为独立提交;
- 将 `search_code/edit_file` 整理为独立提交;
- 确认 `.env.example` 删除是否为预期;
- 修复或平台化处理 Windows `sleep` 单元测试;
- 建立本路线文档的持续更新约定。

M1: Transactional Coding Agent

M1.1 结构化搜索与编辑

状态: 基础版本和 Ripgrep 流式后端已完成。

已完成:

- ✓ Ripgrep JSON 流式后端与 Python fallback;
- ✓ 默认使用 `.gitignore` 规则;
- ✓ 全局结果预算和达到上限时提前终止;
- ✓ Python fallback 移入工作线程, 避免阻塞事件循环;
- ✓ 20 万行和 100 万行合成仓库 benchmark。

后续任务:

- 搜索前后文和多命中;
- 批量 edit 事务;
- 生成统一 diff;

- 所有路径使用统一 WorkspaceBoundary 校验。

M1.2 Git checkpoint、diff 与 rollback

状态：结构化 status/diff 和手动 checkpoint/rollback 基础版本已完成。

已完成：

- ✓ porcelain v2 状态解析，区分 staged、unstaged、untracked、rename 和 conflict；
- ✓ 分支、upstream、ahead/behind 元数据；
- ✓ staged、unstaged、all 三种 diff scope；
- ✓ 字面文件过滤、上下文行配置、二进制标记和硬输出预算；
- ✓ Git 子进程超时、取消、输出溢出和非仓库错误处理；
- ✓ 根 Agent、子 Agent、Planner、Executor、Reviewer 和 Review Skill 接入。
- ✓ 使用临时 index/tree 捕获 tracked、staged、unstaged 和非忽略 untracked 内容；
- ✓ checkpoint 与 Session ID、消息 node ID、run ID 关联；
- ✓ rollback preview、状态确认令牌和 HEAD 变化保护；
- ✓ apply 前自动 undo checkpoint，并在恢复后校验 index/worktree tree；
- ✓ rollback apply 接入 ASK 权限，根 Agent 注册且子 Agent 暂不注册。

后续任务：

- 设计任务开始前和关键步骤的自动 checkpoint 策略；
- 增加 checkpoint list/show、名称查询和 TUI 展示；
- 增加保留期限、引用扫描和 tree object GC；
- 对 metadata 写入、恢复中断和 Git filter 增加故障注入测试；
- 子 Agent worktree 隔离完成后再次开放 checkpoint/rollback；
- destructive Git 操作继续要求明确授权。

M1.3 VerificationManager

状态：项目识别、可审查计划、结构化执行、Parser 和有界自动修复循环已完成。

已完成：

- ✓ 自动识别 Python、Node、Rust、Go 项目；
- ✓ 从标准项目清单、锁文件和 Node scripts 生成验证命令；
- ✓ 统一 test/lint/typecheck/build 阶段顺序；
- ✓ preview/run 两阶段权限模型，执行命令不经过 shell；
- ✓ 每项超时、fail-fast 和头尾输出预算；
- ✓ 最终输出命令、状态、退出码、耗时和关键输出；
- ✓ 根 Agent、Executor 和 Reviewer 工具接入；
- ✓ 建立统一 Diagnostic DTO 和可注册的 ParserRegistry；
- ✓ 解析 pytest、Ruff、Mypy、进程超时/启动错误，并为未知工具提供位置降级；
- ✓ 默认向模型投影结构化诊断并省略重复原始日志，保留显式调试开关；
- ✓ 建立 100 条 pytest/Ruff/Mypy 受控生成语料、独立 Oracle 和字段级评测。

- ✓ 代码写入后标记待验证，并在模型结束前由 Runtime 强制执行真实验证；
- ✓ 验证失败后将结构化诊断反馈给模型，支持“修改→验证→分析→修复→再验证”；
- ✓ 以最大验证次数、总耗时和 AgentLoop 最大步数限制循环，并显式处理权限拒绝；
- ✓ 支持 off/suggest/required_on_write 三种模式及 TOML/环境变量配置；
- ✓ Windows/Python 项目优先使用工作区 `.venv`，避免 PATH 指向缺少测试依赖的解释器；验证子进程禁写 `.pyc`，避免快速同尺寸修复复用旧字节码。
- ✓ 首次内置写操作前由根/子 Agent Loop 自动创建带 session/node/run 关联的任务 checkpoint；
- ✓ 将 Python AST 测试结构索引持久化到 Git 私有目录，并以 worktree tree diff 增量刷新；
- ✓ 支持 changed test、直接 import、传递 import 选择，输出 checkpoint、changed files、selected tests 和原因；
- ✓ 配置/全局文件、未知后缀、解析失败、无映射和并发变化时自动降级全量验证；
- ✓ 集成测试使用一个故意失败的无关测试，证明 Runtime 实际只执行相关 pytest 文件。
- ✓ 在 Click、attrs、ItsDangerous、Pluggy 上建立 detached-worktree mutation benchmark；20 个有效案例均完整复现失败，并由初测暴露/修复非测试 helper、pytest test root 和测试模块继承传播问题。

后续任务：

- 从项目指令安全读取自定义验证命令；
- 增加基于选择比例、历史耗时和索引成本的收益门禁；当前 20 条初测虽为 100% 失败文件召回，但平均仅缩减 3.68% 测试且端到端耗时增加 30.71%；
- 用 coverage 历史和 pytest collection 补充动态 import、fixture 与参数化测试关系，并把当前 mutation 初测扩展为历史 bugfix、跨版本和人工审查留出集；
- 从 BugSwarm/GHLogs 抽样并人工标注真实 CI 留出集，测量跨版本泛化；
- 优先接入 pytest JSON/JUnit、Ruff JSON、Mypy/SARIF 等机器格式，并补 TypeScript、Jest/Vitest、Cargo 和 Go；
- 根据 Bash/MCP/外部进程后的工作区 diff 识别非内置工具变更；
- 跨 Run 持久化验证状态，并增加独立总 token 上限；
- 增加验证历史索引、趋势指标和 TUI 展示；
- 通过进程组或 Sandbox 清理超时命令的完整进程树。

M2: Durable Session Runtime

- ✓ daemon 启动扫描并恢复 Session；
- ✓ active→interrupted 状态迁移并支持继续对话；
- ✓ 旧格式 current_id 迁移；
- ✓ 消息图严格校验与损坏 Session 隔离；
- ✓ 持久化客户端启动目录的绝对 `workspace_root`；
- ✓ 按工作区列出 Session 并按 ID 恢复；
- ✓ 首条用户消息生成 Session 简介，旧会话启动时自动回填；
- ✓ TUI `/resume <number>`、`/new` 与 CLI 序号选择式 `kama resume`；
- ✓ 显式恢复旧版 closed chat，保持 one-shot 不可恢复；
- ✓ 每条 assistant/tool result/compact 逻辑消息实时追加；

- ✓ 恢复 JSONL 已刷盘但 meta 未推进的同 Run 唯一尾节点;
- ✓ 为中断 Run 的未配对 tool use 补充合成错误结果;
- ✓ 可重复的 SessionStore 七阶段 fault hook;
- ✓ 真实 daemon `os._exit`、重启恢复与二次重启幂等性集成测试;
- compact 追加和随机长序列 fault injection;
- 只读诊断和显式修复 API;
- 分支 Head 列表、checkout 和命名;
- history API 与模型投影分离;
- 大型 JSONL 的 `id → byte offset` 索引。

M3: Context Budget 与长期任务

- 模型调用前 token 预检;
- tool result 头尾截断;
- 动态输出预算;
- compact 格式验证;
- 摘要不具备收益时拒绝 compact;
- 1/3/5 次重复 compact 的事实保留评测;
- 结构化 progress/task state 与自由文本 summary 分离。

M4: Secure Execution Runtime

- ✓ WorkspaceBoundary 统一结构化工具路径解析;
- ✓ 绝对路径、父路径和符号链接逃逸防护;
- ✓ 根/子 Agent 继承 Session 工作区, Bash 固定 cwd;
- OS 级文件系统 sandbox;
- 网络默认关闭和域名 allowlist;
- CPU、内存、进程数和超时限制;
- 敏感环境变量过滤;
- PreToolUse/PostToolUse/Stop/PreCompact 等 Hooks;
- 权限策略与 sandbox 形成纵深防御。

M5: Parallel Agent Orchestration

- session 级持久化子 Agent registry;
- 最大并发数;
- 取消传播和任务清理;
- 只读工具并行、写工具串行;
- 工具依赖图;
- 子 Agent 独立 worktree;
- Planner/Executor/Reviewer 的结果交接协议;
- 多 Reviewer 并行检查安全、性能和测试。

M6: Repository Intelligence

- Tree-sitter AST;
- repository map;
- symbol/definition/reference;
- import 和 call graph;
- LSP diagnostics 和 rename;
- 变更影响范围分析;
- 超大型仓库再评估语义/向量检索, 不作为前置条件。

M7: 产品与生态集成

- GitHub Issue→Agent Task→PR;
- PR 评论驱动继续修改;
- CI 失败自动分析;
- Diff-aware inline review;
- IDE/LSP 插件;
- 浏览器与截图验证;
- 多模型路由、fallback 和成本预算;
- 定时任务和后台维护任务。

6. 量化评测体系

6.1 基本原则

任何简历数据都必须满足:

- 优化前保存 baseline commit;
- 优化后保存 optimized commit;
- 使用同一台机器、Python 版本、配置和测试数据;
- 性能实验至少运行 30 次;
- 报告 median/p50 和 p95, 不能只报告最好成绩;
- 原始结果保存为 JSON/CSV;
- 记录测试集生成方式和随机种子;
- LLM 实验固定模型、prompt、温度和最大 token;
- 性能、成本和正确性同时衡量;
- 未实际测量的数据不得写入简历。

建议目录:

```
benchmarks/  
├─ generate_test_log_dataset.py  
├─ generate_session_fixture.py  
├─ benchmark_search_code.py  
├─ benchmark_incremental_tests.py
```

```
|— incremental_tests/{pytest_recorder,coverage_exporter}.py
|— bench_session_recovery.py
|— fault_injection_session.py
|— eval_compaction.py
|— eval_subagent_parallel.py
|— eval_coding_tasks.py
|— datasets/test_logs_v1/
|— results/
|   |— baseline.json
|   |— optimized.json
```

6.2 SearchCode Benchmark

已完成一组热缓存、稀疏固定字符串搜索基准：

环境	语料	Python p50/p95	Ripgrep p50/p95	p50 加 速比
Windows 11、 Python 3.12.13、 Ripgrep 15.2.0	10,000 文件、 1,000,000 行、500 个 命中、预热后 7 轮	3436.772 / 3529.720 ms	413.114 / 467.816 ms	8.319×

本组实验中，默认 Ripgrep 后端相对 Python fallback 的 p50 延迟降低 87.98%，p95 延迟降低 86.75%。原始结果见 [benchmarks/results/search_code_windows_1m_2026-09-14.json](#)，复现命令：

```
uv run python benchmarks/benchmark_search_code.py --files 10000 --lines 100 --  
rounds 7 --max-results 500
```

该结果只代表本机合成语料的预热后稀疏固定字符串场景，不能直接外推到真实多语言仓库、冷缓存、正则或高频匹配场景。

数据规模：

- 1 万、10 万、100 万行代码；
- Python/TypeScript/Rust 混合仓库；
- 精确字符串、正则、稀疏匹配和高频匹配；
- 冷缓存和热缓存分别测试。

指标：

- p50/p95 搜索延迟；
- 扫描文件数和字节数；
- 峰值内存；
- 返回字符/token 数；
- Python 与 Ripgrep 后端加速比；
- 定位正确文件的成功率；
- 单个 Coding Task 的平均搜索调用数和读取文件数。

6.3 Session Recovery Benchmark

构造：

- 1,000、10,000、100,000 个节点；
- 活动链长度为 20、100、500；
- 每隔固定轮数插入 compact；
- tool result 分布覆盖 1 KB、32 KB、256 KB；
- 构造主分支和多个历史分支。

指标：

- 恢复 p50/p95；
- 峰值内存；
- 实际读取字节数；
- 总节点数与活动链长度的增长关系；
- 索引构建和尾部修复耗时。

目标复杂度：

当前：O(JSONL 总节点数)
目标：O(当前活动链长度)

6.4 Crash Consistency Benchmark

在以下阶段注入退出：

1. 节点 write 前；
2. write 后、flush 前；
3. flush 后、fsync 前；
4. fsync 后、meta 更新前；
5. meta 临时文件写入后；
6. replace 前后；
7. compact 追加过程中；
8. assistant/tool 多消息批次中间。

指标：

- 已确认消息丢失数；
- active chain 损坏率；
- 自动恢复成功率；
- 孤儿节点数量；
- 重复恢复的幂等性；
- 1,000 ~ 10,000 组随机故障序列中的失败次数。

2026-09-15 Windows 实测结果：

```
uv run python benchmarks/benchmark_session_crash.py \
  --phases all --ordinals 3 --rounds 3 \
  --output benchmarks/results/session_crash_windows_2026-09-15.json
```

- 环境：Windows 11、Python 3.12.13;
- 矩阵：7 个节点持久化阶段 × 3 轮，在每轮第 3 个节点 (tool result) 硬退出，共 21 次真实 daemon 崩溃;
- 崩溃前先完成一轮对话作为已确认基线，累计检查 84 个已确认节点;
- 自动恢复成功率：100% (21/21) ;
- 已确认节点丢失率：0% (0/84) ;
- 恢复后孤儿节点：0; 重复或未配对 tool result：0;
- daemon 冷重启到可服务：p50 3084.770 ms, p95 3121.508 ms;
- daemon 就绪后的 `session.resume`：p50 9.451 ms, p95 33.470 ms;
- 独立集成测试额外执行第二次 daemon 重启，验证恢复结果幂等。

结果文件：`benchmarks/results/session_crash_windows_2026-09-15.json`。当前数据反映进程硬退出，不等同于断电；正式矩阵集中在 tool result 节点，脚本已支持第 1~4 个 Run 节点，后续应扩大轮数并覆盖 compact 与随机长序列。

6.5 Verification Parser Evaluation

KamaTestLogBench v1 使用生成期已知错误和工具机器格式作为独立 Oracle，实际执行工具而不是手写日志：

环境	数据分布	Oracle	Precision/Recall	原始/结构化字节	字节降幅
Windows 11、Python 3.12.13、pytest 9.1.1、Ruff 0.16.3、Mypy 2.3.1	pytest 40、Ruff 30、Mypy 30	pytest 生成清单+JUnit、Ruff JSON、Mypy 生成清单	100% / 100%	46,298 / 31,402	32.17%

数据集包含 100 份最小可执行项目、原始日志、期望诊断和实际诊断；pytest 另保存 40 份 JUnit XML，Ruff 另保存 30 份 JSON，清理缓存后共 731 个文件、237,684 字节。复现命令：

```
uv run python benchmarks/generate_test_log_dataset.py \
  --output benchmarks/datasets/test_logs_v1 --count 100 --force
```

首次探索性运行只有 30% precision、48% recall，并产生 160 条预测；数据集暴露了 pytest 9 collection summary 格式变化、Windows/POSIX 路径分隔符差异以及 Ruff `[*]` 展示标记。修正后得到 100 条预测并全部匹配。前后结果见 [benchmarks/results/test_log_parser_windows_100_2026-09-15.json](#)；首次运行未绑定独立 baseline commit，因此它用于记录问题发现过程，不冒充外部基准对比。

该结果是同环境、同版本、参与开发的受控生成集上的内部正确性验证，不能外推为真实 CI 泛化准确率；下一步必须加入未参与开发的工具版本、Linux 和人工标注真实日志。32.17% 是 UTF-8 序列化字节降幅，不等同于特定模型 tokenizer 的 token 降幅。

6.6 Incremental Test Selection Evaluation

KamalIncrementalTestBench v1 使用四个真实开源 Python/pytest 仓库的固定 commit，在 detached Git worktree 中进行确定性源码 mutation；原始 clone 只提供已安装的独立虚拟环境，不直接修改。每个候选依次执行：

```
baseline tree → 单点源码 mutation → AST 依赖选择 → 全量 pytest Oracle
→ 增量 pytest → node ID/文件召回对比 → 逐字节恢复
```

pytest 结果由独立插件直接记录 collected、failed、skipped、xfail 和 collection error node ID，不解析终端文本。Runner 仅接受能让至少一个测试失败、没有 collection error 且增量执行本身可完成的 mutation；survived、collection error、timeout/runner error 单独计数。

2026-09-16 Windows 内部初测：

```
uv run python benchmarks/benchmark_incremental_tests.py \
  --project click=D:/git/kamatest/click \
  --project attrs=D:/git/kamatest/attrs \
  --project itsdangerous=D:/git/kamatest/itsdangerous \
  --project pluggy=D:/git/kamatest/pluggy \
  --cases-per-project 5 --candidate-limit 40 --timeout 180 \
  --output benchmarks/results/incremental_tests_windows_20_2026-09-16.json
```

项目	固定 commit	有效/尝试	默认测试数	失败文件召回	完整复现	平均测试缩减	最大测试缩减	平均端到端时间变化
Click	6aabf099bfdd	5/6	2,084	100%	5/5	0.05%	0.05%	+10.32%
attrs	8f767776326f	5/9	1,412	100%	5/5	0.14%	0.14%	+6.85%
ItsDangerous	672971d66a2e	5/7	297	100%	5/5	14.55%	56.23%	+37.49%
Pluggy	0a4974175aa2	5/10	169	100%	5/5	0%	0%	+68.16%

总体 20/20 有效案例完整复现全量 Oracle 的失败 node ID，失败文件 micro recall 为 100%，无全量降级；选择耗时 p50 564.661 ms、p95 805.298 ms。平均测试数量缩减仅 3.68%，选择开销计入后端到端耗时平均增加 30.71%，因此当前结果证明的是“静态选择在小样本上没有漏测”，而不是“已经加速”。

第一次四项目烟雾矩阵曾只有 90% 失败文件召回，并发现三类真实缺陷：把 tests 下 conftest/typing 样例当作执行目标、忽略 pytest test root、测试模块到派生测试模块的传递边中断。修正后重跑 20 条得到上述结果。原始结果见 [benchmarks/results/incremental_tests_windows_20_2026-09-16.json](#)。

为了区分“没有漏测”和“是否多测”，Runner 新增 `--measure-overselection`：普通全量与增量计时完成后，再独立执行一次 `pytest-cov --cov-context=test`，通过 `coverage.py` 公共 `CoverageData.contexts_by_lineno()` API 导出 node ID 到源码行的动态关系，不把 profiling 开销混入原时间指标。相关集合同时按变异行、最小 enclosing 函数/类和整个文件三个粒度计算；`selected - relevant` 作为过度选择，而不是错误地把所有通过测试都视为多测。

2026-09-16 四项目 4 条固定样本覆盖率 smoke:

```
uv run python benchmarks/benchmark_incremental_tests.py \
  --project click=D:/git/kamatest/click \
  --project attrs=D:/git/kamatest/attrs \
  --project itsdangerous=D:/git/kamatest/itsdangerous \
  --project pluggy=D:/git/kamatest/pluggy \
  --cases-per-project 1 --candidate-limit 12 --timeout 300 \
  --measure-overselection \
  --output benchmarks/results/incremental_tests_coverage_4case_2026-09-16.json
```

项目	全量/ 已选 node	enclosing 符号	符号 相关 node	符号 影响 召回	选择精 确率	过度选 择率	文件 相关 node
Click	2,084 / 2,083	BoolParamType	1,120	100%	53.77%	46.23%	1,372
attrs	1,412 / 1,410	astuple	17	100%	1.21%	98.79%	88
ItsDangerous	297 / 130	load_payload	66	100%	50.77%	49.23%	66
Pluggy	169 / 169	is_registered	5	100%	2.96%	97.04%	169

四条 coverage run 与普通全量 run 的失败 node 集合完全一致；符号影响 micro recall 为 100%，选择精确率 31.86%，过度选择率 68.14%。按文件级相关集合计算，选择精确率 44.70%、过度选择率 55.30%。Click 变异行在 coverage 启动前已由导入路径求值，因此行级相关集合为空，但类级和文件级上下文仍有效，这也证明只依赖精确变异行会低估影响。

限制：coverage 相关集合是动态执行近似，不是语义相关性的绝对真值；mutation 和实现共同参与本轮开发，不是独立留出集；coverage 多测评估目前只有每项目 1 个案例；生产选择器仍按测试文件和静态 import 图工作，尚未读取 coverage 索引；全量先于增量运行，存在热缓存顺序偏差。下一步应扩大 coverage 样本、加入收益门禁、随机执行顺序、多轮计时、历史 bugfix 和未参与调试的 held-out 项目，再决定如何将符号级动态关系安全用于生产选择。

6.7 Context Compaction Evaluation

准备 30 ~ 50 组固定长对话，在其中埋入：

- 文件路径；
- 用户约束；
- ID；
- 错误信息；
- 配置值；
- 已完成步骤；
- 未完成 TODO。

分别执行 1、3、5 次 compact，测量：

- `summary_tokens / original_tokens` 压缩比;
- 输入 token 降幅;
- 关键事实 exact-match/semantic recall;
- 后续任务完成率;
- context overflow 次数;
- API 成本;
- 重复压缩后的信息衰减。

6.8 Subagent Evaluation

比较:

- 根 Agent 串行完成;
- 2 个子 Agent;
- 4 个子 Agent;
- 有并发限制与无并发限制;
- 共享目录与独立 worktree。

指标:

- 总墙钟时间;
- 并行加速比;
- 任务成功率;
- token 消耗;
- 写冲突次数;
- 重试次数;
- orphan/cancelled task 数量。

6.9 Coding Task Evaluation

建立小型 `KamaBench`, 优先使用真实 bug 和功能请求:

- 10 个单文件修复;
- 10 个跨文件修改;
- 10 个测试失败修复;
- 10 个重构任务;
- 10 个需要代码搜索和影响分析的任务。

统一指标:

- issue resolved rate;
- tests passed rate;
- regression-free rate;
- patch apply success rate;
- 平均步骤、工具调用、token 和耗时;
- 人工评审发现的严重问题数;

- 未经请求的额外改动行数。

7. 可用于简历的表述模板

以下模板必须在完成真实评测后替换占位符。

Session 与可靠性

设计基于 append-only JSONL 与消息 DAG 的会话存储，通过 `id/parent_id/current_id` 支持累积压缩和分支恢复；引入节点偏移索引后，将 10 万消息会话恢复 p95 从 X ms 降至 Y ms，峰值内存降低 Z%。

实现消息级持久化与崩溃恢复机制，通过 N 组故障注入测试，将已确认消息丢失率由 X% 降至 0，保证 active chain 可验证恢复。

当前可直接使用的基础版表述：

实现 Daemon 启动级 Session 恢复，基于 append-only 消息图重建会话 Head 和锁，将异常退出状态迁移为 interrupted；通过严格校验重复 ID、缺失父节点、循环引用、错误 current_id 和破损 JSON，实现坏会话隔离与合法会话连续恢复。

将客户端启动目录作为 Session 级绝对工作区持久化，设计按工作区过滤和 Session ID 恢复协议，并将统一 WorkspaceBoundary 注入根/子 Agent 的文件、搜索、Git 和验证工具，阻断绝对路径、父路径及符号链接逃逸。

代码搜索与编辑

实现结构化代码检索工具，采用 Ripgrep 流式搜索、Python 自动降级、Gitignore、大文件预检和结果预算；在本机 100 万行合成仓库的 7 轮热缓存测试中，将 p50 搜索延迟从 3437 ms 降至 413 ms，提升 8.32 倍，p95 延迟降低 86.75%。

设计带乐观冲突检测和原子替换的局部编辑工具，将 patch 冲突导致的非预期修改率由 X% 降至 Y%，并支持 checkpoint 级回滚。

基于 Git tree object 构建不污染真实暂存区的 checkpoint/rollback，使用状态令牌和 HEAD 校验阻止并发覆盖，并在恢复前自动生成 undo checkpoint；单元测试覆盖 staged、unstaged、untracked 混合状态的精确恢复与反向撤销。

设计 Session 历史分支机制，将消息图 fork 来源与 Git 代码状态统一关联：沿 `parent_id` 查找最近有效 checkpoint，在成功写操作后生成节点级快照，并以 detached worktree 精确恢复 HEAD、暂存区、工作树和未跟踪文件，实现原会话与实验分支的对话、运行记录和代码状态隔离。

Context 工程

构建面向长时 Coding Agent 的累积上下文压缩机制，在 N 组多轮工具任务中将平均输入 token 降低 X%，关键事实保留率达到 Y%，context overflow 率由 A% 降至 B%。

验证闭环

实现项目感知的自动验证与修复循环，支持测试、lint 和类型检查的失败解析，在 N 项真实代码任务上将最终测试通过率从 X% 提升至 Y%，平均人工介入次数降低 Z%。

当前可直接使用的基础版表述：

构建项目感知的 VerificationManager，自动识别 Python、Node、Rust、Go 工程并生成 test/lint/typecheck/build 计划；在代码写入后的完成阶段由 Runtime 强制验证，将结构化失败诊断反馈给模型执行有界修复重试，并以验证次数、总耗时和 Loop 步数避免失控循环。

基于 Git worktree tree 与任务 checkpoint 构建持久化 Python AST 测试依赖索引，支持 Agent 离线期间变更的增量自愈，并按直接/传递 import 选择相关 pytest；配置变化、解析失败或低置信度场景自动降级全量测试。

安全执行

实现权限策略与 OS sandbox 双层执行边界，覆盖文件、网络和子进程限制；在 N 组越界与提示注入测试中实现 X% 阻断率，同时将安全命令审批次数降低 Y%。

多 Agent

实现 session 级子 Agent 调度、并发限制及 worktree 隔离，在 N 组独立代码分析任务中将端到端耗时降低 X%，写冲突率由 A% 降至 B%。

8. 当前测试状态

截至 2026-09-16:

- `search_code` 单元测试：13 项通过，覆盖两个后端、自动回退、`.gitignore`、用户规则、大文件预检、Unicode 列号和结果上限；
- `search_code/edit_file/Git`、文件过滤、配置传播、权限策略、Runner 和子 Agent 相关定向测试累计：104 项通过；
- Ruff：相关文件通过；
- Mypy：相关源文件通过；
- checkpoint/rollback、工具参数、权限策略和 Runner 定向测试：55 项通过；
- VerificationManager、增量测试索引、自动修复 Controller、Parser、数据集生成器、权限、角色和 Runner 定向测试：102 项通过；
- 消息上下文、Session 存储/恢复、Runner 和 Compactor 定向测试：67 项通过；
- 增量测试索引与 Benchmark Runner 新增回归：8 项通过，覆盖 test root、非测试 helper、继承测试传播、src 候选过滤和 mutation 字节级恢复；
- 最新全量回归：442 项通过、1 项符号链接能力跳过；
- Windows Bash 超时测试已改用当前 Python 解释器执行跨平台 sleep，不再依赖系统是否提供 `sleep` 命令。

SearchCode、Session 崩溃恢复和 Verification Parser 已有可复现结果，可以使用相应小节中的限定版表述；其他能力的延迟、成功率和 token 降幅仍必须等待对应 benchmark 完成后填写。

9. 设计决策记录

2026-09-06：Session 改为统一 append-only 消息图

- 普通消息与 compact 使用同一个 JSONL；
- compact 不覆盖历史；
- 使用 `current_id` 和 `parent_id` 恢复活动链；
- 为未来分支功能保留所有旧节点。

2026-09-06: tool result 截断移出存储层

- canonical JSONL 永久保留完整输出;
- Compactor 只截断发送给摘要模型的副本;
- 手动与自动 compact 使用统一配置;
- prompt 改为累积摘要语义。

2026-09-14: 开始 Transactional Coding Agent 路线

- 新增结构化 `search_code`;
- 新增带冲突检测的原子 `edit_file`;
- 接入权限、角色白名单、Skills、根 Agent 和子 Agent;
- 暂不实现自动 Git checkpoint, 避免当前混合工作树被错误纳入快照。

2026-09-14: SearchCode 默认切换为 Ripgrep 流式后端

- 使用 `rg --json` 获取结构化路径、行号、字节偏移和源代码行;
- 按路径排序, 并在达到全局结果预算时提前回收子进程;
- 将 UTF-8 字节偏移转换成字符列号, 保持两个后端输出一致;
- Ripgrep 缺失或拒绝 Python 兼容正则时透明回退;
- 保留 Python 后端用于跨平台兼容和基准对照。

2026-09-14: Git 只读能力从 Bash 命令升级为结构化工具

- `git_status` 基于 porcelain v2 和 NUL 分隔协议, 路径包含空格时仍能可靠解析;
- `git_diff` 明确区分 staged、unstaged 和 all, 文件参数按 literal pathspec 解释;
- 两项工具默认只读放行, 不要求 Agent 获得任意 Bash 权限;
- 子进程层统一处理超时、取消和硬输出预算, 避免大型 diff 占满模型上下文;
- 暂不实现自动 checkpoint, 避免覆盖当前已有的用户未提交修改。

2026-09-14: 大文件预检与可配置文件忽略

- `read_file` 和 Python 搜索后端在读取内容前先检查文件大小;
- 搜索目录遍历复用 `DirEntry.stat()` 元数据, 避免为每个候选文件增加一次独立路径查询;
- 超过阈值的文件不读取内容, 并用“阈值 + 1 字节”有界读取处理并发增长;
- 新增 `FileAccessConfig.ignore_files` 和共享文件规则匹配器;
- 支持 `.env` 中的 `ignore_file`、`KAMA_IGNORE_FILE` 以及 TOML `[files].ignore_file`;
- 用户规则采用追加和去重语义, 保留原有目录、二进制、文件大小与 GitIgnore 规则;
- 忽略规则统一传递给根 Agent、子 Agent、`read_file`、Ripgrep 和 Python fallback。

2026-09-14: Git tree checkpoint 与两阶段安全恢复

- 使用临时 `GIT_INDEX_FILE` 捕获工作树，checkpoint 过程不修改用户真实 index；
- 独立保存 index tree 和包含非忽略 untracked 内容的 worktree tree，支持还原混合 Git 状态；
- metadata 写入 `.git/kama-checkpoints/`，关联 Session、消息 node 和 run；
- rollback 采用 preview/apply 两阶段协议，apply 必须携带最新仓库状态的 SHA-256 令牌；
- HEAD 改变或 preview 后发生并发文件变化时拒绝恢复；
- apply 前自动建立 undo checkpoint，恢复后重新计算 tree 验证；
- rollback apply 使用 ASK 权限，工具只注册给根 Agent，等待 worktree 隔离后再开放给子 Agent。

2026-09-14: 项目感知 VerificationManager 基础版

- 用标准项目清单识别 Python、Node、Rust、Go，避免由模型自由拼接验证命令；
- 将跨生态检查规范化为 test、lint、typecheck、build 四阶段计划；
- 默认只预览，真正执行时使用 ASK 权限，项目脚本不会因只读工具身份绕过审批；
- 子进程通过参数数组启动而非 shell，单项执行具备超时和有界头尾输出；
- 检查失败作为结构化业务结果返回，不触发工具调用层的自动重试；
- fail-fast 后将剩余项记录为 skipped，保证报告能解释哪些检查未运行；
- 工具接入根 Agent、Executor 和 Reviewer，Planner 不获得执行能力。

2026-09-15: Verification Diagnostic Parser 基础版

- 为每个验证检查显式标记 tool，并以统一 `Diagnostic` 表达类别、严重级别、文件位置、规则码、test ID 和证据；
- `ParserRegistry` 对 pytest、Ruff、Mypy 使用确定性解析器，对 timeout/process error 直接归类，对未知工具仅做保守的位置格式降级；
- 解析失败不调用 LLM 猜测，避免额外 token 成本和不可复现分类；未知失败仍返回最多 4 KiB 原始头尾，防止信息完全丢失；
- `verify_project` 默认只向模型返回结构化诊断，`include_raw_output=true` 可恢复 Runner 已限制在 32 KiB 内的原始输出；
- 8 项 Parser 单测覆盖 pytest failure/collection、pytest 9 summary、Ruff rich/compact、Mypy、进程异常、通用降级、去重和工具推断；当前仓库真实 Ruff 输出冒烟测试提取 16 条诊断；
- 新增可重复生成的 100 条 KamaTestLogBench v1：40 pytest、30 Ruff、30 Mypy，每条保存最小项目、raw log、expected 和 actual；
- 首次评测 precision 30%、recall 48%，据此修复 pytest 9 collection summary、Windows 路径关联和 Ruff 展示标记；修正后受控集 precision/recall 均为 100%，结构化字节减少 32.17%；
- 这些数字不是外部泛化成绩，下一步以未见版本、Linux 和人工标注真实 CI 日志作为 holdout。

2026-09-15：自动验证完成门禁与有界修复循环

- `VerificationController` 观察成功的内置编辑、写入和 `rollback apply`，将当前 Run 标记为 `dirty`；
- System Prompt 会提示 Agent 主动验证，但正确性不依赖提示词：模型尝试结束时，`AgentLoop` 会合成真实 `verify_project` tool call；
- 合成调用仍经过参数校验、权限审批、事件记录和普通工具执行链，不会绕过 `run=true` 的 ASK 边界；
- 验证通过或没有适用检查时允许成功结束；失败诊断作为 tool result 留在上下文，驱动下一轮分析、修复和复验；
- 最大验证次数、墙钟总耗时和 Loop 最大步数共同形成硬上限；权限拒绝、次数耗尽和时间耗尽均给出稳定失败原因；
- 模式支持 `off`、`suggest`、`required_on_write`，并可配置 `max_attempts` 与 `max_total_seconds`；
- 两条真实集成测试在最小 Python 项目中执行编辑和 `pytest`，分别覆盖首次验证通过，以及“失败诊断进入上下文→二次编辑→复验通过”的完整链路；
- 集成测试发现 Windows PATH 可能选中未安装 `pytest` 的系统解释器，因此 detector 改为优先工作区 `.venv`、最后使用当前运行时 `sys.executable`；
- 失败修复用例还发现同一秒内同尺寸改写可能复用旧 `.pyc`，因此验证子进程设置 `PYTHONDONTWRITEBYTECODE=1`，避免一次验证污染下一次验证结果；
- 当前限制是只能观察已知写工具、状态不跨 Run 持久化、没有独立 token 预算，且最终答复的流式展示仍需增加“验证中”状态。

2026-09-15：Checkpoint-aware 持久化测试索引与增量 pytest

- 根 Agent 和具备验证权限的子 Agent 在首次内置写操作执行前自动创建 `verification` checkpoint，普通无写入询问不产生基线；
- 索引保存于 `.git/kama-test-index/index.json`，包含 tree、Python 模块、imports、静态测试 ID 和反向依赖图，不进入 Git status 或任务 diff；
- 启动或验证时用旧索引 tree→当前 tree 的 diff 更新结构，因此 Agent 未运行期间用户修改测试 import 也不会造成永久断层；
- 当前 tree 与任务 checkpoint tree 的 diff 只描述本次任务变化，避免把任务开始前用户已有的 staged/unstaged/untracked 内容误归给 Agent；
- 选择器沿反向图寻找 changed test、直接 import 和传递 import 测试；验证计划保存 baseline checkpoint、changed files、selected tests、原因和 fallback；
- 配置文件、`conftest.py / __init__.py`、非 Python 变化、AST 失败、无映射和索引期间并发变化一律降级全量测试；
- 两阶段 tree 捕获避免索引与实时工作区错位，最新索引 tree 通过内部 Git ref 保留以抵抗离线 Git GC；
- 单测覆盖传递依赖、离线修改后索引自愈和结构配置降级；真实 Loop 集成通过故意失败的无关测试证明 `pytest` 命令只包含相关文件；
- 当前尚未覆盖 Bash/MCP 写入基线、动态 import、fixture 隐式依赖、coverage 历史、非 Git 项目和 Node/Rust/Go 增量选择。

2026-09-15: Session 历史分支、有效 checkpoint 与键盘恢复 UI

- 新增 `session.branch` 和 TUI `/branch [node_id]`, 每个分支都是独立 Session, 拥有自己的 `current_id`、`run`、`compact`、状态与实际工作目录;
- 子 Session 只保存 `forked_from_session_id/node_id/checkpoint_id`, 恢复模型输入时递归读取来源链, 本地 `compact` 仍能截断全部继承历史;
- 当前节点分支会即时捕获代码状态; 历史节点则沿不受 `compact` 截断的完整父链寻找最近 `checkpoint`;
- 成功的潜在写工具在 `tool result` 节点持久化后自动创建 `post-mutation checkpoint`, 普通消息不建快照并继承最近状态;
- 对旧会话增加写入间隙检测: 最近 `checkpoint` 后存在未覆盖的成功写操作时, 拒绝声明可精确恢复;
- 使用 `detached Git worktree` 恢复 `checkpoint` 的 `HEAD`、`index tree`、`worktree tree` 和未跟踪文件, 原目录不执行 `rollback`;
- 每个 `checkpoint` 的 `index/worktree tree` 写入内部 `Git refs`, 抵御 `Git GC`;
- `/resume` 改为方向键或 `j/k` 移动、`Enter` 确认、`Esc` 取消的内联列表, 并保留编号/ID 兼容入口;
- 主要实现: `core/session/{model,store,manager}.py`、`core/git/{checkpoint,worktree}.py`、`core/verification/controller.py`、`core/loop.py`、`core/bus/commands.py`、`core/app.py`、`tui/app.py`;
- 主要测试: `tests/unit/test_session_branch.py`、`test_session_store.py`、`test_git_checkpoint.py`、`test_tui_app.py`、`test_commands_events.py`。

2026-09-14: Daemon 启动恢复持久化 Session

- `Daemon` 在开始监听前扫描 `Session` 根目录并恢复内存索引与逐 `Session lock`;
- `active` 会话被视为上次进程中中断并持久化迁移为 `interrupted`, 恢复后允许继续发送消息;
- `waiting/closed` 状态和原始 `updated_at` 保持不变, 避免重启篡改会话时间语义;
- 旧格式无 ID 节点继续使用稳定虚拟 ID, 并推断、持久化 `current_id`;
- 启动路径严格校验 `meta`、重复节点 ID、`missing parent`、`parent cycle`、`current_id` 和 `JSONL` 行;
- 损坏会话仅从内存索引排除并记录诊断, 不移动或删除用户文件, 也不阻止其他会话恢复;
- 当前只解决已落盘数据的恢复, `Run` 中间消息丢失需要下一阶段实时追加处理。

2026-09-14: 工作区绑定与按 ID 恢复 Session

- 前端以 `Path.cwd().resolve()` 获取启动目录, 作为绝对 `workspace_root` 写入 `Session` 元数据;
- 新增 `session.list` 和 `session.resume`, 列表按工作区过滤, 恢复时再次核对 `Session` 归属;
- `Session` 默认从首条用户消息生成 30 字简介, 启动时为旧版空标题会话从物理消息日志回填;
- CLI `kama resume` 展示当前工作区列表并用序号选择; TUI 支持 `/resume <number>` 与 `/new`;
- TUI/CLI 正常退出不再自动关闭 `chat`, 旧版 `closed chat` 可显式重新打开, `one-shot` 保持终态;
- 新增统一 `WorkspaceBoundary`, 真实路径解析后阻断相对、绝对和符号链接逃逸;
- 文件读写、目录、搜索、`Git`、`Verification`、根 `Agent` 和子 `Agent` 统一继承 `Session` 工作区;

- Bash 仅固定执行 cwd，任意 shell 命令的完整文件系统隔离留给 OS sandbox；
- 协议文档由 Pydantic 模型重新生成，单元与双进程 IPC 测试覆盖工作区过滤和恢复。

2026-09-15: Session Run 改为消息级实时持久化

- `ExecutionContext` 通过可选 append sink，在完整 assistant 响应、每个 tool result 和 compact 节点形成时同步持久化；
- `SessionStore` 对单个节点执行“追加 JSONL、flush、`fsync`、原子更新 meta Head”，Runner 结束时只补写尚未持久化的节点；
- 多个并行工具结果以独立物理节点保存，恢复模型输入时合并相邻 tool result，兼顾崩溃粒度和 Anthropic 消息约束；
- daemon 启动时将同一 Run、当前 Head 之后的唯一 durable orphan 纳入活动链，处理 JSONL 已落盘但 meta 尚未推进的崩溃窗口；
- 中断 Run 中未配对的 tool use 会收到合成 `is_error` 结果，避免恢复后裁掉后续消息或产生非法模型历史；
- canonical 日志只记录完整逻辑消息，不保存尚未完成的流式 assistant 草稿，避免恢复半条结构化 tool call。

2026-09-15: RunManager 与有界 Git checkpoint

- 将 Core 原先无 ID 的 `_running_runs: set[Task]` 替换为统一 `RunManager`，顶层 chat、one-shot 和子 Agent 均以 `run_id/session_id/workspace/parent_run_id` 注册；
- 新增 `agent.list_runs`、`agent.cancel`、`agent.snooze` RPC，取消操作按绝对工作区校验，父 Run 默认级联取消后代；
- RunManager 依据总耗时、阶段耗时和最后进展时间检测慢任务；等待权限不判定卡死，提醒默认只询问，不自动终止；
- TUI 新增 `Ctrl+C` 取消当前 Run、`Ctrl+R` 活动列表，以及 `/runs`、`/cancel [run_id]`；活动列表支持方向键与 Enter，慢任务提醒支持取消或延后五分钟；
- Session 取消后持久化为 `interrupted`，已生成的消息节点和工作区修改保留，可通过 resume 继续；
- Git checkpoint 不再因 `git status/diff/log/show/branch` 等明确只读 Bash 触发；内置 edit/write 使用精确路径，未知写入仍采用保守策略；
- worktree tree 从真实 index tree 起步，已跟踪变化使用 `git add -u`；未跟踪路径先执行 1,000 文件、20 MiB 单文件、100 MiB 总量预算，再分批写入临时 index；
- Bash 和 Git 子进程使用独立进程组，取消/超时时在 Windows 通过 `taskkill /T`、POSIX 通过进程组信号清理完整子进程树；
- `git_status` 默认返回条目从 500 降为 100，summary 仍基于完整有界状态，减少路径列表占用模型上下文；
- 新增/更新测试覆盖按 ID 取消、父子级联、提醒不自动关闭、工作区隔离、海量未跟踪文件快速拒绝、scoped checkpoint、unborn repository 暂存文件和 TUI 列表；截至 2026-09-16 的全量结果为 442 passed、1 skipped。

2026-09-15: 真实 Daemon 崩溃一致性评测

- `SessionStore` 增加仅通过依赖注入启用的七阶段 fault hook, 覆盖节点 write/flush/fsync 和 meta 临时文件/replace 边界, 生产路径默认无行为;
- `CoreApp` 支持注入隔离 Session 根目录、确定性 Provider 和 fault hook, 使 benchmark 使用真实 IPC、Agent Loop 与 daemon 进程且不依赖外部模型;
- 每轮 Run 开始前先持久化 `active` 状态和 run ID, 保证第二轮对话中断也能进入启动恢复逻辑;
- 正式矩阵首次发现 durable tool result 恢复后重复追加合成错误结果的问题, 修正为 Head 快进后立即持久化, 再检查未配对 tool use;
- 新增真实 `os._exit` 集成测试并执行二次重启, 验证已确认消息、工具配平、孤儿节点和恢复幂等性;
- Windows 21 次正式硬退出矩阵实现 21/21 自动恢复、0/84 已确认节点丢失, `session.resume` p50 9.451 ms、p95 33.470 ms。

2026-09-16: 真实项目增量测试 mutation 初测

- 新增安全 Benchmark Runner: 固定 commit、detached worktree、原项目独立解释器、单点 mutation、pytest node ID Recorder、全量 Oracle、逐字节恢复和临时 worktree/ref 清理;
- 初始 4 条烟雾矩阵仅有 90% 失败文件召回, 直接暴露测试辅助文件误选、pytest test root 不一致和继承测试传播中断; 修正后增加针对性回归测试;
- 扩展到 Click、attrs、ItsDangerous、Pluggy 共 20 个有效 mutation, 20/20 完整复现失败, 失败文件 micro recall 100%;
- 当前平均测试缩减仅 3.68%, 选择 p50/p95 为 564.661/805.298 ms, 端到端平均慢 30.71%, 说明纯静态 import 图需要 coverage 关系和收益门禁, 当前不得包装成“增量测试加速”。
- 增加测试级 coverage context 多测评估; 四项目固定 smoke 的符号影响 micro recall 为 100%, 但选择精确率仅 31.86%、过度选择率 68.14%, 量化证明当前策略“安全优先但明显保守”, 并暴露精确行覆盖会漏掉导入期求值影响。

2026-09-16: 语言无关语义 diff 分层与 PythonParser

- 将原先耦合在 `semantic_diff.py` 中的 AST 逻辑拆为 `DiffAnalyzer`、`ChangedRanges`、`ParserRegistry`、`PythonParser`、`UnifiedSymbol` 和 `ChangeClassifier`; 后续 JS/TS、Go 只需实现同一 parser 协议并注册文件后缀;
- checkpoint 新增零上下文 patch 和 tree blob 读取接口, 语义分析始终基于 baseline/current tree, 不直接读取可能并发变化的工作树; 大于 1 MiB 的源码不进入语义解析;
- 仅当新旧 Python AST 能匹配同一函数/方法、签名和装饰器未变、修改行位于函数体时按 import 绑定缩小 pytest node; 类属性、模块变量、签名/装饰器/继承、重命名、删除、解析失败等保留静态文件选择或全量回退;
- 同一测试文件中 `result/other` 的定向回归验证已从文件级选择缩小为 `tests/test_service.py::test_result`; 四项目语义 smoke 仍 4/4 完整复现失败, 当前固定样本多为高风险核心变化, 因此没有冒险宣称端到端加速。

2026-09-17: 可复现 Agent Evaluation Harness

- 新增 `kama eval` 和 JSON suite 契约，将真实 Agent 执行、外部 oracle、指标采集和证据留存统一成标准评测生命周期；
- 每次任务解析并记录固定 commit，在独立 detached worktree 中运行，Agent 写入不会修改原工作区；任务结束、失败或超时后自动清理，支持显式保留现场；
- oracle 以 `argv` 数组无 shell 执行，复用 `VerificationRunner` 的超时、有界输出和结构化 Parser；
- `EventBus` 自动聚合步骤、工具调用/失败、权限请求和 Token，suite 输出成功率、验证通过率及耗时 p50/p95；
- 使用 checkpoint 的有界临时 index 捕获包括未跟踪文件在内的最终 tree，生成 changed files 和最大 8 MiB 的二进制 patch；
- 评测 artifacts 独立于 Session 根目录，不污染 `/resume`；关闭用户级 context 注入，保留版本库内项目 context；
- 主要实现：`core/harness/{models,metrics,evaluation}.py`、`cli/commands/eval.py`、`core/runner.py`；示例为 `examples/evaluation_suite.example.json`；
- 定向回归覆盖仓库隔离、oracle、patch、指标、超时和 worktree 清理；当前尚未运行真实模型任务矩阵，因此不填写成功率或性能提升数据。

10. 后续更新规则

以后完成 Coding Agent 相关功能时，在本文件同步更新：

1. 更新“当前能力快照”和能力矩阵状态；
2. 在对应 Milestone 标记完成项和遗留项；
3. 记录关键设计选择及其原因；
4. 注明主要实现入口、核心实现文件和对应测试文件；
5. 写明新增或修改的测试；
6. 如果有 benchmark，记录环境、样本量、p50/p95 和原始结果路径；
7. 更新可用于简历的表述，但不得填写未经测量的数据；
8. 更新文档顶部“最近更新”日期。

建议每项功能使用以下记录模板：

日期：
问题：
Baseline:
设计：
关键实现：
相关文件：
测试：
Benchmark:
结果：
限制与下一步：
简历表述：

11. 最近下一步

1. 整理现有工作树和提交边界；

2. 增加 Session 只读诊断和显式修复 API;
3. 扩展 compact 与随机长序列故障注入, 并增加断电语义测试;
4. 为 Verification Parser 增加真实 CI/跨版本/Linux 留出集, 并补机器格式及 Node/Rust/Go 解析器;
5. 基于现有 20 条 mutation 初测加入收益门禁、coverage/pytest collection 关系、历史 bugfix 与 held-out 项目, 并扩展到至少 100 个有效案例和多轮随机顺序计时;
6. 为 checkpoint 增加 list/show UI、保留期限、分支 worktree 清理和容量配额;
7. 为 `search_code` 增加搜索前后文和单行多命中;
8. 增加结构化 Git log, 以及可视化 Session 消息树/node/checkpoint 关联查询与历史节点选择器。